

Benchmark Harness (BENCH_HARNESS)

Version: 2.3.0 | Updated: 2026-07-19

Also ingested into the help corpus, so you can ask AIStudio how to run benchmarks. Companion reading: the audited evidence in `BENCH - Canonical Suite - README` and `Synthesis` (same folder), and how to read a benchmark in *TUTORIAL §5*.

AIStudio ships with a benchmark harness (`benchmarks/bench.py`) that runs a structured question set against any corpus and produces timestamped reports. It is designed for the demo corpus out of the box but works with any corpus and any YAML question file you provide.

Reports are written to `benchmarks/<corpus>/reports/` as paired `.md` and `.json` files, named by corpus and timestamp.

Quick Start (Demo Corpus)

Make sure the backend is running (`ais_start`), then:

```
ais_bench
```

This runs all 14 demo questions against the `demo` corpus and writes a timestamped report to `benchmarks/demo/reports/`.

The questions themselves are a plain YAML file you can read and edit:

```
benchmarks/demo/demo_questions.yaml
```

Every corpus follows the same convention — `benchmarks/<corpus>/<corpus>_questions.yaml` for the default set, with named variants under `benchmarks/<corpus>/questions/`. Open the demo file to see what a question looks like; the fields are documented in **Question File Format** below, and **Bringing Your Own Corpus** walks through writing one from scratch.

CLI Reference

```
ais_bench [OPTIONS]
```

What to run

Option	Meaning
<code>--corpus</code> NAME	Corpus to query. Default <code>demo</code> .
<code>--scope</code> NAME	Restrict to a named subset of the corpus, e.g. <code>lang_en</code> . Reads <code>data/corpora/<corpus>/scopes/<corpus>_<scope>_scope.yaml</code> .
<code>--questions</code> STEM	Named question set, given as a stem, not a path . Empty uses the default set; a stem resolves to <code>questions/<corpus>_<stem>_questions.yaml</code> . An unknown stem is a hard error — never a silent fallback to the default set.
<code>--canonical</code>	Reproduce the audited reference runs exactly, each on its pinned model. See The reference suite below.
<code>--batch</code>	Run the reference sets against <i>this</i> machine — every set twice by default (smallest installed model + largest that fits). Narrow with <code>--model</code> .

How to run it

Option	Meaning
<code>--model</code> NAME	Ollama model. Default is whatever the API is configured to use. With <code>--batch</code> , also accepts <code>largest</code> or <code>smallest</code> .
<code>--top-k</code> INT	Chunks retrieved per query. Default 10 for the financial corpora; a corpus with no stored default falls back to 5.
<code>--temperature</code> FLOAT	Sampling temperature, 0.0–2.0. Default 0.3.
<code>--api</code> URL	Backend URL. Default <code>http://localhost:8000</code> .
<code>--fit-policy</code> MODE	What to do when the chosen model won't fit in memory: <code>skip</code> , <code>downshift</code> (use the largest that fits), or <code>force</code> (run anyway — it will likely hang). Omit it and the harness asks, offering the models that do fit. Needs a terminal; a fully non-interactive run fails closed rather than guessing.

Output and inspection

Option	Meaning
<code>--dry-run</code>	With <code>--batch</code> or <code>--canonical</code> , print the resolved run set and a runtime estimate, then exit without executing. Use it before committing hours.
<code>--mem-track</code>	Show a free-memory column per question, in GB. On by default; <code>--no-mem-track</code> hides the column. The end-of-run memory recap and the JSON record are written either way.
<code>--emulate-ram GB</code>	Run as if this machine had <code>GB</code> of RAM, e.g. <code>24</code> . Reserves the difference so the shortage is real and every memory check sees the smaller machine — the only way to reproduce a constrained Mac's behaviour on a larger one. Released when the run ends. Minimum 8 GB. The report filename records it (<code>..._m4max-24gb-emu_...</code>), so an emulated run is never confused with a real machine of that size.
<code>--full</code>	Put full answers in the report instead of the first four paragraphs.
<code>--no-markdown</code>	Skip the <code>.md</code> report (JSON only).
<code>--help / --version</code>	Show help, or the harness version.

Question File Format

Questions are YAML files organized by topic. Each question has an `id`, a `question` string, optional `keywords` (a soft signal feeding the GREEN/AMBER/RED rating — see Scoring below, not a binary gate), and optional `notes`:

```
- topic: Architecture Methodology
  questions:
    - id: my_first_question
      question: What is the relationship between business strategy and technology strategy?
      keywords: [business, strategy, technology]
      notes: Core intellectual thread – business strategy drives technology strategy.

    - id: my_second_question
      question: How do you design an IT organization around architectural principles?
      keywords: [organization, architecture, principles]
```

Place question files at `benchmarks/{corpus}/{corpus}_questions.yaml` for auto-detection. See `benchmarks/demo/demo_questions.yaml` for a fully annotated example.

Bringing Your Own Corpus

1. Create and ingest your corpus

Create a corpus via the UI, upload your documents, and ingest them.

2. Write a question file

Create `benchmarks/{corpus}/{corpus}_questions.yaml` following the format above.

3. Run the benchmark

```
ais_bench --corpus my_corpus
```

Reports are written to `benchmarks/my_corpus/reports/`.

Running Systematic Benchmarks

The reference suite — `--canonical` VS `--batch`

Most of this page is about running *one* thing and looking at it. The reference suite is the opposite: a fixed set of runs, defined once, that you can re-run whenever something changes. It lives at `benchmarks/batch/bench_canonical.yaml`, and each entry names a corpus, a question set, a scope, and the retrieval parameters — so "run A" always means the same thing.

Why a fixed set exists at all. Benchmarks are only useful comparatively. A score of 8/10 means nothing on its own; 8/10 *against the same questions the previous release scored 6/10 on* means something. Pinning the runs — same corpora, same questions, same depth — is what turns individual numbers into a trend. The audited figures in

`BENCH - Canonical Suite - README and Synthesis` all come from this set.

Why there are two commands instead of one. The suite has to serve two goals that pull against each other.

Reproducibility says: pin everything, including the model, so a number from today can be compared to one from three months ago. That is `--canonical` — it runs each entry exactly as written, on its pinned model, and it is how the published figures are regenerated.

But the pinned model is not universal. The reference model is a 27B, which needs roughly 32 GB of unified memory; on a 24 GB Mac it will not load at all. Taken literally, that means a constrained machine cannot run the reference suite — and a suite nobody can run is not

reproducible in any useful sense. It also leaves the more practical question unanswered: *what does the machine in front of me actually do?*

That is `--batch`. It runs the same fixed set, but resolves the model at runtime against what this machine has installed and can hold:

```
ais_bench --canonical           # exact reproduction, pinned models
ais_bench --batch               # this machine: smallest installed AND largest that fits
ais_bench --batch --model largest # only the largest that fits
ais_bench --batch --model smallest # only the smallest installed
ais_bench --batch --model gemma3:12b # a model you name
ais_bench --batch --dry-run      # show what would run, execute nothing
```

Bare `--batch` runs every entry **twice** — once on the smallest model you have, once on the largest that fits — because the interesting question on a given machine is usually not "how good is this model" but "how much does the model tier matter here". Two runs per entry give you that range directly.

Before you start one. Bare `--batch` is eight runs and can take hours. The harness prints an order-of-magnitude estimate before it begins, and `--dry-run` shows the fully resolved run list — including which models were selected — without executing anything. Use it first.

Reports from either mode are written per run to `benchmarks/<corpus>/reports/` and name their model, so runs from different tiers never get confused with each other.

Use the individual flags below to explore outside the fixed set.

Compare models

```
ais_bench --model gemma3:27b # the SEC 10-K benchmark model
ais_bench --model llama3.1:8b
ais_bench --model llama3.1:70b
```

Compare retrieval depth

```
ais_bench --top-k 3
ais_bench --top-k 5
ais_bench --top-k 10
```

Compare temperature

```
ais_bench --temperature 0.0
ais_bench --temperature 0.3
ais_bench --temperature 0.7
```

Each run produces a separate timestamped report in `benchmarks/demo/reports/`:

```
benchmarks/demo/reports/benchmark_demo_2026-04-15_21-30.md
benchmarks/demo/reports/benchmark_demo_2026-04-15_21-30.json
```

Reading a Report

Each report contains:

Configuration — full snapshot (corpus, model, top-k, temperature, timestamp, **machine** and **context window**) so every report is self-describing and reproducible. The machine block names the chip, physical RAM, and any emulated target; `num_ctx` is read from the running backend rather than the client's environment, because exporting it after `ais_start` changes nothing and the two can disagree.

Memory is reported in GB, never as a percentage. A percentage is a share of *physical* RAM, so under `--emulate-ram` it describes memory the run cannot see — "10% free" on a 128 GB box emulating 30 GB is 12.8 GB, which is ample. GB is the same number on every machine. The reload threshold is likewise the model's own footprint (~10 GB for a 12B, ~21 GB for a 27B), not a fixed share of the box: per-question working memory scales with parameter count, because KV cache scales with layers × heads.

Summary — total questions, the  rating tally, binary pass rate (back-compat), average latency.

Infrastructure — vector store, embedding model, reranker, corpus stats.

Results table — per question: latency, rating () , citation density, sources, notes.

Detailed results — full answer text with inline citations, per question.

The first query in a cold session is slow (20–50s) while the LLM loads into memory. Subsequent queries run at ~6–7s warm on M4 Pro/Max, or ~26–28s on M4 Air — latency is hardware-bound, not corpus-bound. The benchmark summary reflects warm latency once the first query completes.

Scoring — the GREEN/AMBER/RED rating

`evaluate()` returns a tri-state `rating ∈ {GREEN, AMBER, RED}` as the primary verdict, with the older binary `pass` demoted to a back-compat soft signal:

-  **GREEN** — cited, right firm, honest, keywords present, adequate citation density.

- **AMBER** — cited and plausibly right, but with a soft weakness (a keyword miss, low citation density, or partial entity coverage) → read up / audit.
- **RED** — no citations, honest-empty, or wrong firm (entity coverage 0 with a filter active).

Two ratings beyond the colour scale.

● **BLOCKED** — the fit guard declined to run the question because the model would not fit in the memory free at that moment. The model never ran, so this is a *machine* event, not a *quality* one: BLOCKED questions are excluded from the pass-rate denominator and from the latency average, and the summary names them with the remedy (free memory, or use a smaller model). Detection requires both the guard's advisory text and a sub-second latency, so a genuinely fast answer is never misread as a block.

Groundedness, and the "uncited but grounded" re-rating — a zero-citation answer used to score RED, indistinguishable from a wrong or invented one. Since reports now carry the retrieved chunks, the harness measures how much of the answer's vocabulary actually appears in that context. Above ~70% the answer is re-rated **AMBER, "uncited but grounded"**: right answer, missing attribution. Below ~40% it is flagged as possible fabrication. This matters because the two are genuinely different defects and were being reported as the same one — the same question can score RED with zero citations on one run and GREEN with six on the next, on identical claims from the same model and corpus — sometimes with the *uncited* answer marginally better grounded. Without this check the grader measures tag emission and reports it as correctness.

Groundedness is deliberately crude — lexical overlap, not entailment. It cannot prove an answer faithful. It can separate one built from the retrieved context from one built from nowhere, which is the distinction that changes the rating, and it makes the audit below automatic rather than manual.

Keyword matching is demoted from the verdict to one input among several — a GREEN answer can miss a keyword and an AMBER one can hit them all. The thresholds are v1 defaults, calibrated against the audited canonical runs; the rating is a triage signal, not a grade. The contract is still the audit: verify the cited chunk, never the colour alone.

SEC 10-K Corpus

To benchmark against the full 101-filing SEC corpus (21 firms):

```
# Download first (one-time, ~5 min); then ingest via the UI Upload button
ais_download_sec_10k

# Then benchmark
ais_bench --corpus sec_10k          # K=10 is the stored default for sec_10k
```

Default question set ships at `benchmarks/sec_10k/sec_10k_questions.yaml` ; named subsets (e.g. `frontier`) live in `benchmarks/sec_10k/questions/` .

Reports are written to `benchmarks/sec_10k/reports/` .

What's Coming

- **Cross-run comparison** — diff two reports on latency, answer quality, source overlap (partially delivered via `--batch` + the canonical suite)
- **Hardware metadata** — machine specs recorded in report header for M4 Air vs M4 Max comparisons
- **Model quality evaluation** — human eval framework for 8b vs 70b answer quality

See `docs/PRODUCT_ROADMAP.md` for the full roadmap.

*** **